

THINKFREE FINANCE

Hosting & Deployment Guide

Step by step: host the app, connect it to a server,
and run it from the GitHub repo as the backend

Covers local run, data build, GitHub deploy (Render / Railway),
Cloudflare, scheduled data refresh, and the pre launch security checklist

Contents

ThinkFree Finance: Hosting and Deployment Guide

1. What you are deploying
2. Prerequisites
3. Step 1: Get the code on your machine
4. Step 2: Configure secrets
5. Step 3: Build the gated datasets
6. Step 4: Run it locally and verify
7. Step 5: Put the code on GitHub
8. Step 6: Deploy the GitHub repo as the backend (Render)
9. Step 7: Put Cloudflare in front
10. Step 8: Keep the data fresh (scheduled refresh)
11. Step 9: Post deploy verification
12. Pre launch security checklist
13. Operations
14. Troubleshooting

ThinkFree Finance: Hosting and Deployment Guide

This is a step by step guide to hosting ThinkFree on the internet, connecting it to a server, and running it straight from your GitHub repository as the backend. It is written for this specific app (a FastAPI backend in `server/` plus a static front end in `webapp/`), so every command and setting matches what is actually in the repo. Follow the steps in order.

1. What you are deploying

ThinkFree has two parts that ship together:

- A **FastAPI backend** (`server/`) that handles login, sessions, and the gated data API, and also serves the front end shell.
- A **static front end** (`webapp/`) that the browser runs.

The recommended setup is **single server, same address**: one web service serves both the page and the `/api/*` endpoints from the same origin. The browser then makes only same origin calls, so there is no CORS to configure and the session cookie just works.

The front end is **dual mode** and detects its environment automatically:

- **Server mode**: when the FastAPI backend answers `GET /api/healthz`. Login goes to `/api/auth/*` and the datasets come from `/api/data/*` only after login. The raw data files and API keys are never sent to the browser. This is what you run in production.
- **Static mode**: when the page is opened as a plain file with no backend. A soft localStorage gate is used. This is a development fallback only and is not real security.

```
Browser -> https://your-domain (Cloudflare) -> Host (Render/Railway)
                                              |
                                              FastAPI (server/)
                                              - serves webapp/ shell
                                              - /api/auth/* (login, MFA)
                                              - /api/data/* (gated datasets)
                                              |
                                              private_data/*.json (built offline)
                                              users.db             (SQLite, on a disk)
```

2. Prerequisites

Before you start, have these ready:

- A **GitHub account** and the ThinkFree repository pushed to it (private is fine and recommended).
- **Python 3.10+** installed locally.
- A **host account**. This guide uses **Render** as the primary path and lists

Railway as an alternative. Both can build and run a GitHub repo directly.

- A **Cloudflare account** (free tier is enough) and a **domain name** if you want

a custom address and the web application firewall.

- Your **data pipeline API keys** if you want to refresh the datasets (LegiScan,

Open States, Finnhub, FEC, Census, BEA, BLS, OpenAI, FRED). The app will run without refreshing, serving the last built data.

3. Step 1: Get the code on your machine

```
git clone git@github.com:<your-username>/ThinkFreeFinance.git
cd ThinkFreeFinance
```

If the repo is already local, just make sure it is up to date:

```
git pull origin main
```

4. Step 2: Configure secrets

Secrets live only in environment variables, never in the code. Create a local `server/.env` from the example and fill it in:

```
cp server/.env.example server/.env
# Generate a strong signing key and paste it into TF_SECRET_KEY:
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

Edit `server/.env` and set at minimum:

- `TF_SECRET_KEY` : the long random string you just generated. The server refuses

to start in production with the placeholder, by design.

- `TF_BETA_KEY` : your closed beta invite key. Rotate this before launch because an

earlier value was exposed in git history.

- `TF_PRODUCTION` : keep `0` for local testing. Set `1` only on the host.
- `TF_ALLOWED_ORIGINS` : leave as the local default for now. In production set it to

your real origin, for example `https://your-domain.com`.

`server/.env` is gitignored and must never be committed. On the host you will set these same values in the dashboard instead of using a file.

5. Step 3: Build the gated datasets

In server mode the app reads datasets from `private_data/`, not from the public `webapp/js/*_data.js` files. Build them once:

```
python3 scripts/extract_data_to_json.py
# writes private_data/*.json and private_data/manifest.json
```

This converts the existing `webapp/js/*_data.js` bundles into the gated JSON the backend serves. If `private_data/` is empty, the data endpoints return a 503 with a clear message telling you to run this script.

To pull fresh data from the live sources instead of reusing the last build, run the build scripts in `webapp/` (for example `build_news_intel.py`, `build_prices.py`) with your API keys set, then run the extractor again. Automating this on a schedule is covered in Step 7.

6. Step 4: Run it locally and verify

Use a clean virtual environment. Do not reuse the old `tf_env`.

```
python3 -m venv server/.venv
server/.venv/bin/pip install -r server/requirements.txt
server/.venv/bin/uvicorn main:app --app-dir server --reload --port 8000
```

Open `http://localhost:8000`. You should see the login gate. Create an account with your beta key, take the short profile quiz, and confirm the dashboard renders.

Quick checks that gating works (run in a second terminal):

```
curl -i http://localhost:8000/api/data/bundle # expect 401 (not logged in)
curl -i http://localhost:8000/js/data.js      # expect 404 (raw data blocked)
curl -i http://localhost:8000/api/auth/me    # expect 401
curl -i http://localhost:8000/healthz       # expect 200
```

If those behave as described, the backend is working and you are ready to host it.

7. Step 5: Put the code on GitHub

The host runs your code straight from GitHub, so the repo is your backend source.

1. Confirm the sensitive files are gitignored and not tracked:

```
git status --ignored
git ls-files | grep -Ei "\.env|users\.db|auth_seed|private_data/[^\.]" # should print
nothing
```

1. Commit and push:

```
git add -A
git commit -m "Prepare for deployment"
git push origin main
```

A private repository is recommended. The host connects to it with read access and redeploys automatically whenever you push to `main`.

8. Step 6: Deploy the GitHub repo as the backend (Render)

Render builds and runs the repo directly. This is the connect to a server step.

1. Sign in to Render and choose **New, Web Service**.

2. **Connect your GitHub account** and pick the ThinkFree repository. Render will redeploy on every push to the branch you choose (use `main`).

1. Configure the service:

- **Environment:** Python 3.
- **Build command:**

```
pip install -r server/requirements.txt
```

- **Start command:**

```
uvicorn main:app --app-dir server --host 0.0.0.0 --port $PORT
```

The host injects `$PORT`; do not hardcode 8000 in production.

1. Add the environment variables under the service **Environment** tab (not a committed file):

- `TF_SECRET_KEY` = your strong random key
- `TF_BETA_KEY` = your invite key
- `TF_PRODUCTION` = `1`
- `TF_ALLOWED_ORIGINS` = `https://your-domain.com` (or the Render URL for now)
- plus any data pipeline keys you use

1. Add a **persistent disk** so the user database and built data survive restarts

and redeloys. Mount it and point the data at it:

- Mount path example: `/data`
- Keep `users.db` and `private_data/` on that disk. The simplest approach is to

build `private_data/` during the build step, or upload it once to the disk. Note: SQLite on a single instance is fine for a closed beta. Before scaling past one instance, move to managed Postgres (the `server/db.py` module is the single place to swap the store).

1. Build the data on the host. Either add the extractor to the build command:

```
pip install -r server/requirements.txt && python3 scripts/extract_data_to_json.py
```

or upload a prebuilt `private_data/` to the persistent disk.

1. Click **Create Web Service**. Render clones the repo, installs, and starts

uvicorn. When the deploy is live, open the Render URL and confirm the login gate appears and `GET /healthz` returns 200.

Railway alternative

Railway works the same way: New Project, Deploy from GitHub repo, then set the start command to `uvicorn main:app --app-dir server --host 0.0.0.0 --port $PORT`, the build to `pip install -r server/requirements.txt`, add the same environment variables, and attach a volume for `users.db` and `private_data/`. Fly.io is also viable with a small Dockerfile that runs the same uvicorn command.

9. Step 7: Put Cloudflare in front

Cloudflare gives you HTTPS, a content delivery network, and the security layer that makes the backend safe for a beta.

1. Add your domain to Cloudflare and update your domain registrar to use the Cloudflare name servers.

1. Create a DNS record (usually a CNAME) pointing your domain or subdomain at the host address from Render or Railway. Keep it proxied (the orange cloud).

1. In Cloudflare SSL/TLS, set the mode to **Full (strict)** and enable

Always Use HTTPS.

1. Turn on the **web application firewall** managed ruleset and **Bot Fight Mode**.
2. Add a **rate limiting** rule on `/api/*` to slow credential stuffing and abuse.

This sits on top of the per endpoint limits the app already enforces.

1. Optional but recommended: add **Cloudflare Turnstile** to the signup and login forms for bot protection.

Once the domain resolves through Cloudflare, set `TF_ALLOWED_ORIGINS` on the host to that exact origin and redeploy.

10. Step 8: Keep the data fresh (scheduled refresh)

Important: the app serves the data from the last build. It does not scrape the web on its own. To get fresh news and prices, the build pipeline has to run on a schedule on a machine that has network access and your API keys.

The simplest automation is a **GitHub Actions** workflow that runs on a cron, rebuilds the data, and triggers a redeploy:

1. In the repo, add your API keys as **repository secrets** (Settings, Secrets and variables, Actions). Never put them in the workflow file.

1. Create `.github/workflows/refresh-data.yml` with a schedule (for example every morning). The job checks out the repo, installs the pipeline requirements, runs the relevant `webapp/build_*.py` scripts using the secret keys, runs `scripts/extract_data_to_json.py`, and then either commits the regenerated `private_data/` or calls the host deploy hook so the new data goes live.

1. Verify the first scheduled run produces changed data and that the live site

reflects it after the redeploy.

If you want more articles per run, expand the source list inside `webapp/build_news_intel.py` (add feeds and raise the per source limits) before scheduling. This is the piece that turns the app from a static snapshot into a daily updating feed.

11. Step 9: Post deploy verification

After the site is live behind your domain, confirm:

- Visiting the domain shows the login gate over HTTPS.
- You can create an account with the beta key and reach the dashboard.
- `https://your-domain/api/data/bundle` returns 401 when not logged in.
- `https://your-domain/js/data.js` returns 404 (raw data blocked).
- The browser developer tools Network tab shows data arriving from `/api/data/*`

and shows no API keys anywhere in the responses or the scripts.

- Logging out and back in works, and the theme and profile persist.

12. Pre launch security checklist

- [] `TF_SECRET_KEY` is a fresh 48 character or longer random value.
- [] `TF_PRODUCTION` is `1` so cookies are Secure and HSTS is sent.
- [] You rotated `TF_BETA_KEY` and every data pipeline API key that was ever

shared or committed.

- [] No secret string appears under `webapp/` :

```
grep -rEi "api[_-]?key|secret|token" webapp/
```

- [] `server/.env` , `users.db` , and `private_data/` are not in git:

```
git status --ignored
```

- [] Cloudflare web application firewall and rate limiting are on for `/api/*` .
- [] A backup of `users.db` exists, or you have moved to managed Postgres.
- [] The persistent disk is attached so data and accounts survive restarts.

13. Operations

- **Backups:** snapshot `users.db` regularly. For more than a small beta, migrate to managed Postgres.

- **Logs and monitoring:** ship the application audit events to an external log sink so they survive restarts and can be alerted on. The admin audit endpoint reports whether the tamper evident log chain still verifies.

- **Scaling:** SQLite and the in memory rate limiters assume a single instance.

Before running multiple instances, move the user store to Postgres and the rate and cadence counters to a shared store such as Redis.

- **Redeploys:** pushing to `main` triggers an automatic redeploy on the host.

14. Troubleshooting

- **Login gate appears but data never loads:** `private_data/` is empty or the manifest is missing. Run `scripts/extract_data_to_json.py` on the host, or upload a prebuilt `private_data/` to the persistent disk.
- **Server refuses to start in production:** `TF_SECRET_KEY` is still the placeholder. Set a strong value.
- **Endless redirect loop:** a proxy is not forwarding the original scheme. Make sure the host or Cloudflare is in front and `TF_PRODUCTION` is `1`.
- **Login works locally but not in production:** check `TF_ALLOWED_ORIGINS` matches your real origin exactly, and that the site is served over HTTPS so the Secure cookie is accepted.
- **Data looks identical day to day:** the scheduled refresh in Step 7 is not set up or did not run. The app serves the last built data until the pipeline runs again.